

approach can test and analyse their system even before any code is written. That is, testing is integrated with every stage of the lifecycle, right from cross-checking the design with the requirements to simulating the system to see how it would work on the targeted hardware. This is one of the biggest advantages of using a modelling-cum-simulation approach, unlike the earlier models where the very first integration between hardware and software happened in real time at a very late stage, and subsequent discovery of errors proved to be very expensive.

Moreover, since the models are expressed using graphical block diagrams, algebraic equations and other easy notations, these can be easily understood by all the parties involved. The requirements are very clear to all those involved. This reduces the communication gap between the client and systems and software engineers. This also makes it easy to specify changes to the requirements as and when they arise.

Paul Raistrick describes in a recent Embedded.com article how model-based development also reduces ambiguity throughout the process: "Software engineers develop models that describe architectural, structural and behavioural aspects of the software. They typically reference control law algorithms from the systems engineering environment. However, it may not be clear to a software engineer the implicit assumptions that the control model was built on. Therefore it is easy for software engineers to make incorrect assumptions about the model behaviour. If the software is based upon different assumptions from those of the control model, the behaviour of the software will diverge from the required behaviour of the model. The control model is precise from the system engineer's point of view but ambiguous from the software engineer's point of view." Such problems are eliminated when you use a modelling tool, be-

cause each component has a specific meaning, which does not lend itself to interpretation by various parties. The specifications are automatically converted into code by the software itself, thereby reducing another level of potential misinterpretation.

Another very significant advantage of model-based tools is component reuse. Since the system is designed as functional blocks, it is very easy, even for the user, to identify and reuse common modules retrieved from earlier projects.

Putting two and two together, it is obvious that model-based development also saves a lot of time, effort and money that would otherwise be spent in designing the system using old-fashioned methods, writing complex code, trying to make sure that everybody understands what is expected of him, sorting out communication gaps, redoing a lot of work when errors are discovered and so on. In today's dynamic business environment, there cannot be a stronger point in favour of model-based development!

How is it done?

To design and develop an embedded system using the model-based approach, first you need to choose a modelling tool.

Most modelling tools are based on, or are variants of, the Object Modelling Group's unified modelling language (UML 2.0) and the more recent system modelling language (SysML) standards.

UML is a visual language for specifying, constructing and documenting the artefacts of systems. You can use it with all processes, throughout the development lifecycle, and across different implementation technologies. SysML extends UML to efficiently develop more complex, heterogeneous systems.

Systems and software modelling suites such as IBM Rational Rhapsody, The MathWorks' MATLAB and Simulink, National Instruments'

LabView and Artisan's Studio Uno automate the usage of UML and/or SysML, spanning the entire process from requirements gathering and design right up to simulation testing and even automatic generation of code. In short, these modelling tools cater to the developers' urgency to complete a system before their competitor does.

You can choose a toolset depending on the purpose of the embedded system. For example, for aircraft-related systems or other safety-critical projects, developers might prefer SCADE, or for digital signal processing systems, they might opt for LabView. Other factors to be considered include platform or real-time operating system compatibility, support for multiple processors and distributed systems, Eclipse integration, real-time emulation and code generation capabilities, ability to import and use legacy systems, and so on.

Focus on testing

At one point of time, when modelling tools had just come about, developers thought of these as merely a support system for documentation at various stages of the development lifecycle. However, they soon realised that the role of models goes beyond documentation, to construction and validation of the system.

Today, the most acknowledged benefit of modelling tools is in testing. No other approach makes it so easy to avoid and detect errors.

"Products and systems are becoming increasingly complex. They require a high amount of testing. And in many cases you need to adhere to several norms (such as for safety-critical systems), and also show a certification authority that your systems have been built according to the legal norms. Basically, the increasing complexity across different domains has led to the demand for tools that help developers design products more innovatively and in a hassle-free fashion," says Prashant Rao, technical manager, The MathWorks India. Companies like The